

10 | K8s 极简实战（五）：如何将集群的业务服务暴露外网访问？

王炜 · 云原生架构与GitOps实战



你好，我是王炜。

上一节课，我们学习了 Env、ConfigMap 和 Secret 三种管理应用配置的方法，了解了它们的适用场景。

当我们将应用顺利迁移到 Kubernetes 之后，接下来我们面临的第一个问题是，用户如何访问集群内部的业务服务呢？因为无论是 Pod 还是 Service，它的 IP 地址都是集群内的虚拟 IP 地址，也叫做 VIP，它实际上是一个内网 IP 地址，只能提供集群内的访问能力，并不能在公网环境下进行访问。

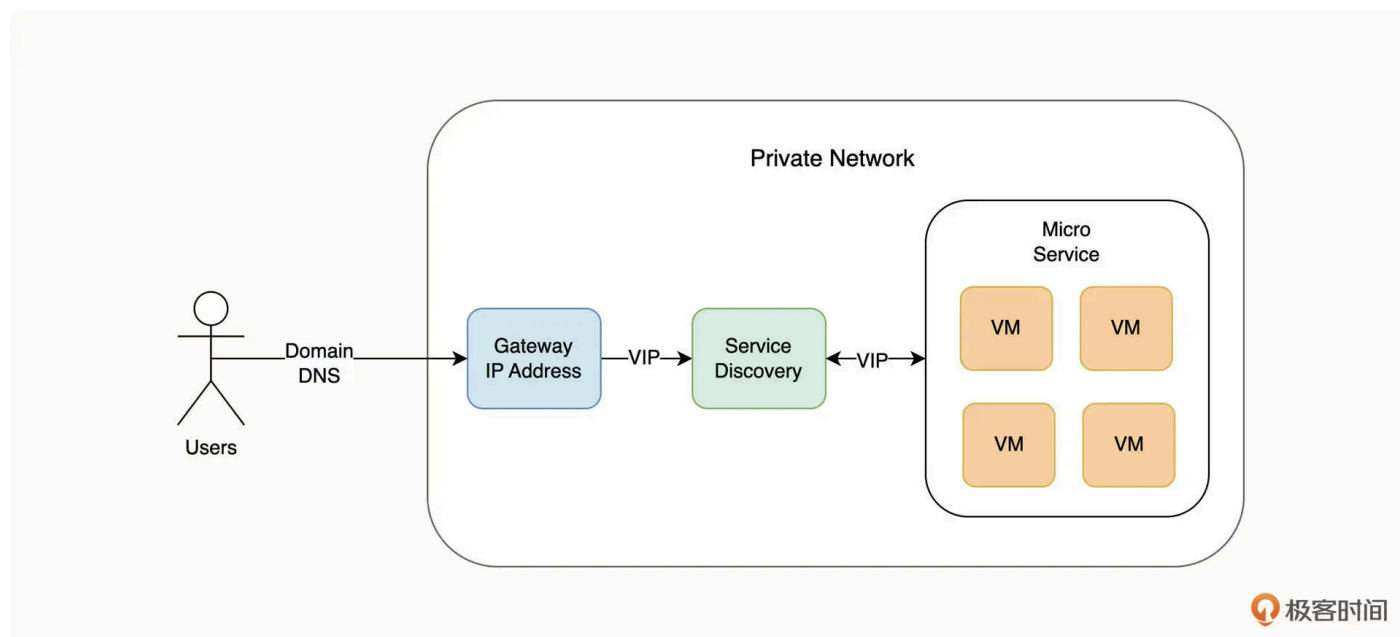
这节课，我们先来温习一下传统应用暴露公网的方式，然后结合示例应用，进一步学习如何对外暴露 Kubernetes 集群内的应用服务。

在开始之前，你需要确保已经按照 [第 5 讲](#)“示例应用介绍”的引导在本地 Kind 集群部署了示例应用。

传统应用的服务暴露

我们先来回顾一下传统的微服务应用是如何对外暴露的。

一般来说，一个典型的微服务应用在系统的最外层会使用网关或者负载均衡器作为系统的入口，然后，根据路由规则和服务发现机制将流量转发到实际的后端微服务中（一般是业务进程所在的虚拟机上）。整体架构如下图所示。



在这个典型的微服务架构中，**网关是系统唯一的入口**。它将通过一个外网 IP 暴露业务系统，除了网关以外，整个业务系统的所有服务都在私有网络下，彼此通过 VIP 进行通信，外部无法访问除了网关以外的任何服务。通常，由于用户访问业务系统一般是使用域名，所以在网关前面还会有 DNS 解析步骤。

显然，在这种架构体系下，对外暴露业务只需要赋予网关服务一个公网 IP 地址就可以达到目的了。

Kubernetes 服务暴露

那么，在 Kubernetes 里面有没有类似的机制呢？结合 Kubernetes Service 对象，如果我们能赋予 Service 一个公网 IP，是不是就可以暴露 Service 选择器所关联的 Pod 了呢？

这个思路完全正确，但我们怎么才能让 Service 获得一个公网 IP 呢？

在回答这个问题之前，我想先请你回想一下 [🔗第 8 讲](#) 的内容。在讲解如何解决服务发现问题时，我有提到过 Service 的两种类型，我有提到过 Service 的两种类型，分别是 **NodePort** 和 **Loadbalancer**。这两种类型都可以为 Service 赋予公网 IP。

接下来，我们深入了解一下这两种 Service 类型。

NodePort

当 Service 被配置为 NodePort 类型之后，Kubernetes 会在每一个节点上监听指定的端口（一般是 30000–32767），当通过节点的公网 IP + 端口号的形式访问时，请求会被转发到对应的 Service 当中。你可以理解为，NodePort 类型可以直接让 Service 在节点层面对外暴露。

在之前部署示例应用时，我们在本地为 Kind 集群安装了 Ingress-nginx 组件，在 Kind 环境下，这个组件其实就是通过 NodePort 的方式暴露的。你可以通过 `kubectl get service` 来获取 Ingress-nginx 的 Service Manifest。

 复制代码

```
1 $ kubectl get service ingress-nginx-controller -n ingress-nginx -o yaml
2 apiVersion: v1
3 kind: Service
4 metadata:
5     .....
6     name: ingress-nginx-controller
7     namespace: ingress-nginx
8 spec:
9     .....
10    ports:
11    - appProtocol: http
12      name: http
13      nodePort: 31844
14      port: 80
15      protocol: TCP
16      targetPort: http
17    - appProtocol: https
18      name: https
19      nodePort: 32606
20      port: 443
21      protocol: TCP
22      targetPort: https
```

```
23 selector:
24     app.kubernetes.io/component: controller
25     app.kubernetes.io/instance: ingress-nginx
26     app.kubernetes.io/name: ingress-nginx
27 type: NodePort
```

在这段 Manifest 内容中，我们重点关注 Selector 字段，还有 Ports 字段下的 **NodePort**、**Port** 和 **TargetPort**。

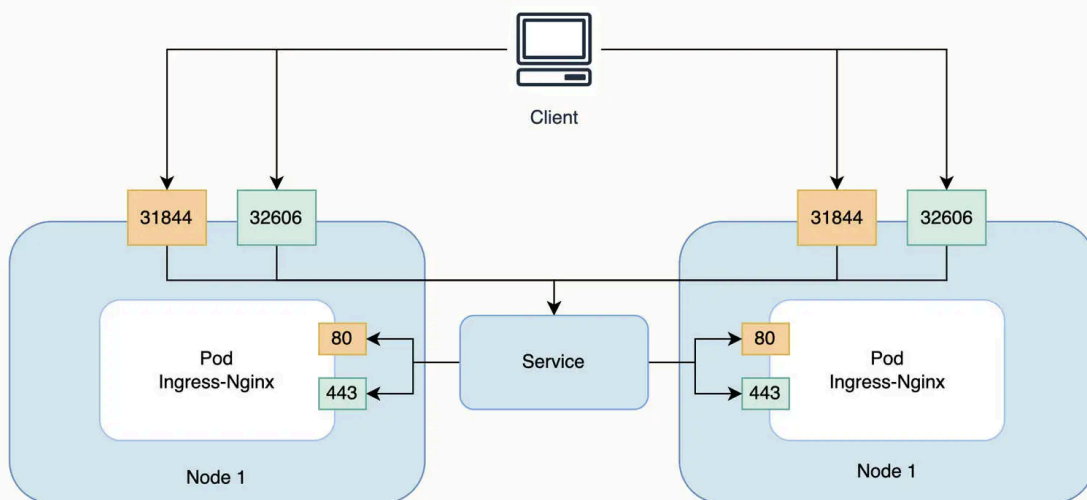
Selector 是选择器，它将匹配 Pod 模板里的 Labels，作用是抽象一组 Pod 服务并将流量在这些 Pod 中做负载均衡。

Ports 字段下定义了两个数组。

第一个数组中的 Port 字段代表 Service 的访问端口，你可以理解为，是 Service 在集群内部的暴露端口。TargetPort 表示目标端口，作用是告诉 Service 将请求转发到 Pod 的哪个端口，你可以理解为，是业务进程在容器里的监听端口。显然 Nginx 在容器内的监听端口是 80。最后也是最重要的 NodePort 字段，它表示需要将服务暴露在 Kubernetes 节点的什么端口，这里具体的含义是将服务暴露在 Kubernetes 节点的 31844 端口上。

同理，Ports 的第二个数组也是代表类似的含义。

最后，这段 Service Manifest 实现的效果是，访问 Kubernetes **任何一个节点**的公网 IP+31844 端口或 32606 端口，请求流量都会被转发到 Ingress-Nginx Pod 的 80 端口或 443 端口上。



NodePort 的暴露方式虽然可以直接复用 Kubernetes 节点的公网 IP，但我并不推荐你在生产环境使用它。主要的原因有两个。

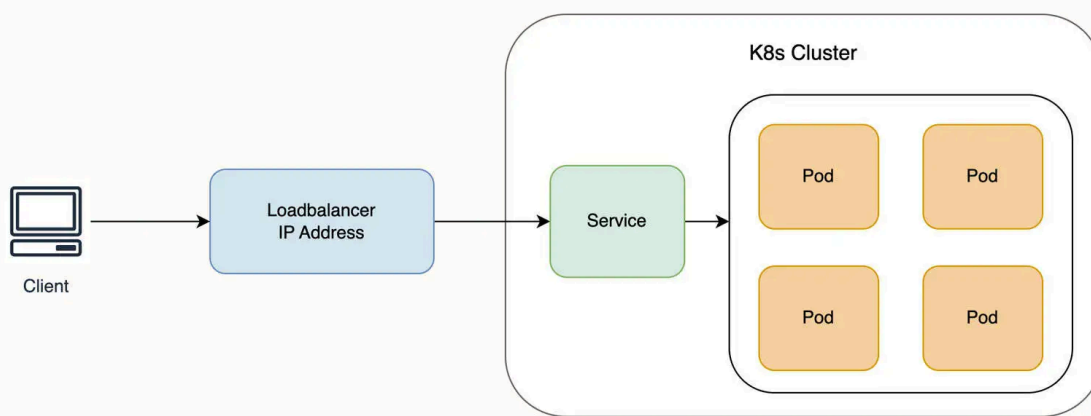
首先，直接对外暴露服务不利于统一管理外部请求流量。

其次，一个端口只能绑定一个服务，并且默认的端口范围是有限的，所以在较大规模场景时使用容易产生端口冲突。如果你希望临时访问集群内的业务服务，建议你使用端口转发进行访问，它适用于大多数的临时场景。

Loadbalancer

除了 NodePort 类型，Loadbalancer 类型也可以对外暴露 Service 服务，也就是我们常说的负载均衡器类型。

Loadbalancer 类型一般依赖于云厂商实现。当 Service 被声明为负载均衡器类型时，云厂商会创建一个负载均衡器实例并和集群的 Service 关联，借助负载均衡器的外网 IP 地址，实现 Service 的对外暴露。此时，相当于每一个 Loadbalancer 类型的 Service 都具有一个外网 IP 地址，所有流量先通过负载均衡器，再转发到对应的 Service 当中，如下图所示。



Loadbalancer 类型相比较 NodePort 有一定的优势。比如，理论上来说它暴露服务的数量不会受到端口数量的限制。从架构设计上来说，暴露服务和 Kubernetes 节点实现了解耦，是一个非常不错的选型。

需要注意的是，每声明一个 Loadbalancer 类型的 Service，都会创建一个新的负载均衡器实例，负载均衡器由于具有固定 IP 地址，所以费用也相对较高，并且还需要为流量额外付费。所以，在实际的项目中，我们一般不直接用 Loadbalancer 类型对外暴露服务，而是通过**网关**来实现服务暴露，这和我们之前提到的传统应用的服务暴露方式非常类似。这时候，就不得不提到 Ingress 了。

Ingress

Ingress 是 Kubernetes 的一个内置对象，通常我们把 Ingress 看作 **Service 之上的 Service**。Ingress 对象只用来声明路由策略，并不处理具体的流量转发。要使得 Ingress 生效，我们还需要额外安装 Ingress-Controller，例如 Ingress-Nginx。

在生产环境中，Ingress-Nginx 一般就是以 Loadbalancer 类型来对外暴露的，Ingress-Nginx 实际上充当的是网关的角色，这样做的好处是，**我们只需要一个负载均衡器实例，通过路由策略，就可以对外暴露所有的业务服务。**

示例应用的 Ingress

接下来，我们通过示例应用来进一步理解 Ingress 对象。在部署示例应用时，我们已经为本地集群部署了 Ingress-nginx，并且将 Ingress 对象部署到了集群中，你可以通过 `kubectl get ingress` 来获取 Manifest。

 复制代码

```
1 $ kubectl get ingress frontend-ingress -n example -o yaml
2 apiVersion: networking.k8s.io/v1
3 kind: Ingress
4 metadata:
5     .....
6     name: frontend-ingress
7     namespace: example
8 spec:
9     ingressClassName: nginx
10    rules:
11    - http:
12        paths:
13        - backend:
14            service:
15                name: frontend-service
16                port:
17                    number: 3000
18            path: /?(.*)
19            pathType: Prefix
20        - backend:
21            service:
22                name: backend-service
23                port:
24                    number: 5000
25            path: /api/?(.*)
26            pathType: Prefix
```

在这段 Ingress 配置中，我们重点关注 Paths 字段下的 Path 和 Backend 字段。

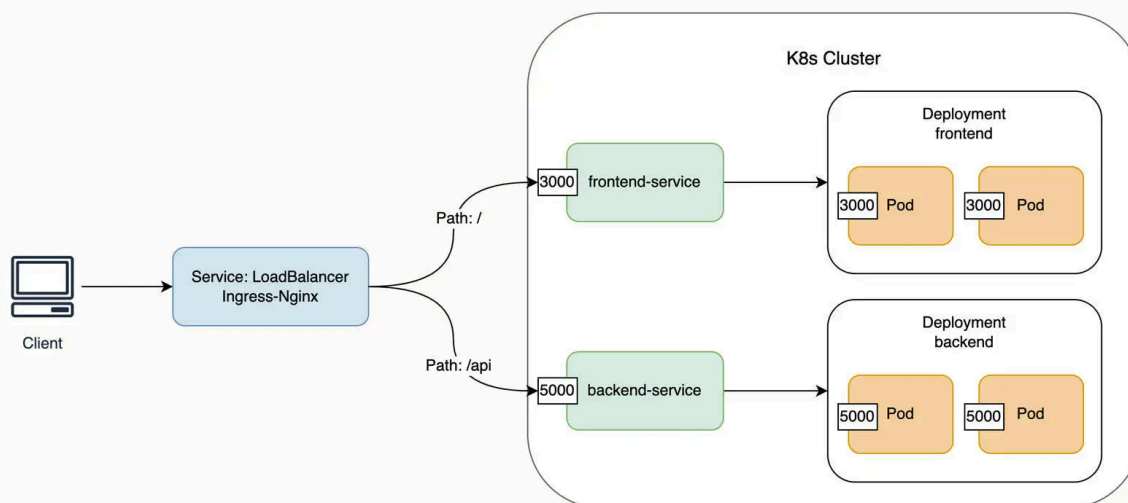
Paths 字段下有两个数组。第一个数组代表的路由策略是，当 URL 包含 / 前缀匹配时，那么将请求转发到 Backend 字段的配置的服务中，也就是转发到 frontend-service 的 3000 端口。同理，第二个数组的含义是，当 URL 包含 /api 前缀匹配时，那么将请求转发到 backend-service 的 5000 端口。

细心的你应该会发现，Ingress 指定的 Service 端口号其实就是 Service 对象的 Port 字段，这里我们可以结合 frontend-service 的内容进行对比。

复制代码

```
1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: frontend-service
5 spec:
6   type: ClusterIP
7   selector:
8     app: frontend
9   ports:
10  - port: 3000
11    targetPort: 3000
```

也就是说，以 Paths 第一个数组的路由策略为例，当 Service 接收到 Ingress 转发过来的流量之后，Service 会继续将流量转发到符合选择器 Labals app=frontend Pod 的 3000 端口上，这样就完成了一个完整的请求链路。为了让你更好地理解，我画了张流量的链路图，你可以结合图例来进行理解。



极客时间

你需要额外注意一个细节，在本地的 Kind 测试集群和示例应用中，Ingress-Nginx 是通过 NodePort 的方式对外暴露的，这是因为我们在本地 Kind 集群中安装的是特殊版本的

Ingress-Nginx，而生产版本的 Ingress-Nginx 一般是通过 LoadBalancer 对外暴露的。

生产环境下部署 Ingress-Nginx

通常，在生产环境下，我们会使用云厂商直接提供的 Kubernetes 集群和部署生产版本的 Ingress-Nginx。要部署生产版本的 Ingress-Nginx 可以使用下面的命令。

 复制代码

```
1 $ kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/con
```

部署完成后，你可以通过 `kubectl get svc` 来获取 Ingress-Nginx 的外网 IP 地址。注意，需要指定 `ingress-nginx` 命名空间。

 复制代码

```
1 $ kubectl get svc -n ingress-nginx
2 NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      P
3 ingress-nginx-controller            LoadBalancer        10.96.146.9      18.176.38.12     8
```

其中，EXTERNAL-IP 就是 Ingress-Nginx 的外网 IP 地址，这个 IP 也就是业务系统的唯一入口，我们可以将它配置到 DNS 域名解析的记录中，这样用户就可以通过域名的方式来访问业务应用了。

在多数实际项目中，Ingress 都是服务暴露的最佳实践。

总结

在这节课中，我们回顾了传统微服务应用的暴露方式。也就是，在最外层部署一个网关，把它作为业务系统的唯一入口。这种服务暴露方式具有很多优点，例如可以很方便地统一管理进入业务系统的流量，网关层面也可以做一些统一的认证和授权等。

在 Kubernetes 环境下，对外暴露服务则需要通过 Service 来实现。由于 Service 的 IP 默认是一个集群内的 IP，无法从外部访问，所以我们需要为 Service 赋予外网 IP 地址。其中，

NodePort 类型可以通过 Kubernetes 节点外网 IP + 端口号的方式提供外网访问能力，不过并不推荐在生产环境使用这种方式。

LoadBalancer 类型则需要依赖云厂商的负载均衡器，一般由云厂商实现。在对 Service 配置为 LoadBalancer 类型后，云厂商将会异步创建负载均衡器实例，这种方式将暴露服务和 Kubernetes 节点进行了解耦，是一种常用的服务暴露方式。

不过在生产环境下，我们也不推荐以 LoadBalancer 的方式暴露所有需要在外网访问的业务服务，因为这不利于统一管理访问流量，并且还要为多个负载均衡器实例支付高昂的费用。为了解决这个问题，我们引入了 Ingress 来暴露服务。

值得注意的是，要使用 Ingress 除了声明 Ingress 对象以外，还需要为集群安装 Ingress-Controller，例如最常见的 Ingress-Nginx。在生产环境下，Ingress-Nginx 正是通过 LoadBalancer 类型的 Service 来自身暴露在公网环境的。

通过 Ingress 暴露服务的方式是我们在生产环境下最常用的方法，同时也是服务暴露的最佳实践。

思考题

最后，给你留两道思考题吧。

1. 你能简单分享目前项目使用的服务暴露方式吗？可以使用文字描述也可以尝试画一个架构图。
2. 请你将目前工作中业务服务的暴露方式和 Kubernetes Ingress 服务暴露方式做一个简单的比较。你认为它们的优劣势分别是什么？

欢迎你给我留言交流讨论，你也可以把这节课分享给更多的朋友一起阅读。我们下节课见。

AI智能总结

本文介绍了在Kubernetes集群中业务服务如何暴露外网访问的重要问题。首先从传统应用的服务暴露方式出发，介绍了Kubernetes中的NodePort和Loadbalancer两种Service类型，以及Ingress对象的作用。NodePort类型通过在每个节点上监听指定端口实现对外暴露，但在生产环境中并不推荐使用。Loadbalancer类型依赖云厂商实现负载均衡器，具有较大的服务暴露数量与Kubernetes节点的解耦优势，但费用较高。文章强调了

Ingress对象的重要性，作为Service之上的Service，通过安装Ingress-Controller实现路由策略，对外暴露所有业务服务。在生产环境中，Ingress-Nginx通常以Loadbalancer类型对外暴露，充当网关角色，通过路由策略实现对外暴露。这些内容为读者提供了在Kubernetes集群中实现业务服务对外暴露的多种方式和相应的优缺点，有助于读者快速了解Kubernetes集群中业务服务暴露的技术特点。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (17)

最新 精选



李多

2023-01-08 来自广东

这里推荐一个开源项目：MetalLB。之前在做实验的时候，拉取的微服务yaml很多都用了LB。但是这些通常需要租用云厂商的k8s集群才能有。就找到了 MetalLB 这个项目，支持在本地集群、裸金属集群上面部署LB。在开发测试或者实验环境也能使用 LoadBalancer 类型的服务暴露了。我在用下来感觉配置也是比较简单，好上手。

<https://github.com/metallb/metallb>

<https://metallb.universe.tf/>

作者回复: 非常不错的项目~

共 4 条评论 >

👍 9



GAC·DU

2022-12-30 来自广东

老师，域名只能绑在一个IP上，只使用ingress做为流量入口会造成单点故障。如果在ingress前面再加一个LB，域名绑在LB上，利用LB将流量分发到ingress上，这样可以吗？

作者回复: 可以这么做。

不过在生产环境下，一般 ingress-nginx HA 我们会这么做：

1. 在几台高配节点上部署独占的实例，并配置相同的资源配额和限制，这样可以避免业务系统抢战资源以及 ingress-nginx 被驱逐。
2. 为 ingress-nginx 配置 HPA 策略

这样就可以得到高可用网关。

共 3 条评论 >

👍 8



小路飞 😊

2023-01-30 来自新疆

我这边的云平台，采用traefik

作者回复: 也是非常不错的技术选型，Traefik 也非常成熟了。



👍 2



农民园丁

2023-01-14 来自内蒙古

在腾讯云搭建了1个master节点，2个node节点的实验环境。

➔ ingress git:(master) kubectl get svc -n ingress-nginx

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
AGE				
ingress-nginx-controller	LoadBalancer	10.97.238.89	<pending>	80:3244
6/TCP,443:31186/TCP	18m			
ingress-nginx-controller-admission	ClusterIP	10.105.63.105	<none>	443/TC
P				

外网IP始终是pending?

请问老师，ingress-nginx-controller 各个云服务商都是部署这个链接的yaml文件吗：

<https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.3.1/deploy/static/provider/cloud/deploy.yaml>

作者回复: 部署的 ingress 版本看起来没问题，可以尝试 kubectl describe service ingress-nginx-controller -n ingress-nginx 查看一下事件。另外可以去腾讯云控制台打开负载均衡器，检查实例初始化的情况。

共 4 条评论 >

👍 2



农民园丁

2023-01-12 来自内蒙古

如果是Kind创建的cluster，就是从宿主机的80端口-暴露-的？

作者回复: 是的，前提是需要指定 config.yaml 暴露 80 端口。



👍 2



gxxc

2023-05-24 来自上海

有几个问题请教：

1. nodePort的ingress-nginx-controller是监听了3万之后的端口，但外部用户只会使用默认的80/443访问，所以需要有一个网关把流量从80/443发送到对应的nodePort上。在自建集群里，除了在最前端增加HAproxy等反向代理，还有什么其他办法么？

2. 虽然大家都在说“ingress是最前端的网关”，但实际好像并不是这样，ingress好像只是ingress-nginx处理路由的配置文件，正如文中所说的“ingress只用来声明路由策略”。

从ingress-nginx看，对外的nodePort是SVC-ingress-nginx-controller暴露的。那么流量路径应该如下：

客户流量 ----> NodePort (SVC-ingress-nginx-controller) ----> 路由处理 (POD-ingress-nginx-controller) ----> 对应服务 (SVC-frontend-service) ----> 真实负载 (POD-frontend-6b48fbbc48-ks9br)

如果上述流量路径正确，那么可以看出实际上仍然只有SVC和POD，只不过人们把处理路由的网关称为ingress。

这个路径里有2个节点是需要HA的，第一是POD-ingress-nginx-controller，第二是真实负载POD。所以老师在第一个评论里回复是“为 ingress-nginx 配置 HPA 策略”，对吗？

作者回复: 非常好的问题。

第一点，对于云厂商的集群，可以直接使用 LB 类型来暴露 Ingress-Nginx。对于自建集群，可以考虑使用 MetalLB + LB 的方式来暴露。

第二点，其实大家这么说的原因主要是因为实际生产上 Ingress-Nginx 一般都是以 LB 方式暴露的，这样的话流量的入口是直接从 LB 打到 Ingress-Nginx Pod 上，然后进行转发，所以说它是流量入口。NodePort 的暴露方式是因为实验环境没有 LB 的支持，所以用该方式代替，你画的流量链路可

以这么理解。

最后关于 2 个节点需要 HA 这块我没理解，可以补充一下吗？

共 2 条评论 >

👍 1



JDLYyear

2022-12-31 来自广东

老师，文中说：Ingress 对象只用来声明路由策略，并不处理具体的流量转发。
那流量转发实际上还是service来做吗？

作者回复: 这里我们安装的是 Ingress-Nginx，流量实际上先是由Nginx 转发到 Service 的，然后 Service 转发到业务 Pod 里的。



👍 1



Ock0

2022-12-30 来自广东

生产环境下部署 Ingress-Nginx，公网 ip 是如何分配给 Ingress-Nginx ？

作者回复: 在生产环境下部署的 ingress-nginx 是 LoadBalancer 类型的 Service，云厂商会自动创建负载均衡器并进行关联。



👍 1



橙汁

2022-12-30 来自广东

文章中演示的在生产环境部署ingress，感觉大家不理解别纠结 看看云厂商的比如阿里 有组件部署后会直接与slb链接，自动生成IP地址，直接配置ingress规则即可，我猜老师演示的也是在云服务器上不然不会出来公网地址。

作者回复: 👍 是的，生产环境下云厂商会实现 lb 类型的 ingress，所以就会有外网 IP，本地 kind 集群是没有的。

共 2 条评论 >

👍 1



fan

2023-05-17 来自广东

传统微服务架构

nginx（有外网IP） -> gateway(内网IP) -> nacos -> 业务微服务

k8s中，不需要gateway 这个微服务了吗？

还是该怎么理解，谢谢老师

作者回复: 如果是使用 K8s 的服务发现，就不需要内网的 gateway 了，如果服务发现有专门的服务，那就还是需要 gateway。

